MedIntel – Smart Health Monitor

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

AMAN SHARMA **EN23CS301119**

AKSHAT YADAV **EN23CS301106**

AMAN SAHU **EN23CS301118**

Under the Guidance of

Prof. DIGENDRA SINGH

Prof. ASHISH SHRIVASTAV

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

MedIntel – Smart Health Monitor

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

AMAN SHARMA	AKSHAT YADAV	AMAN SAHU
EN23CS301119	EN23CS301106	EN23CS301118

Under the Guidance of

Prof. DIGENDRA SINGH

Prof. ASHISH SHRIVASTAV

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work **"**MedIntel – Smart Health Monitor**"** is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled **"**MedIntel – Smart Health Monitor **"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, completed under the supervision of **Prof. Digendra Singh** and **Prof. Ashish Shrivastav**, Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student with date:

_____

AMAN SHARMA – EN23CS301119
AKSHAT YADAV – EN23CS301106
AMAN SAHU – EN23CS301118

B.Tech. III Year
Department of Computer Science & Engineering
Faculty of Engineering, Medicaps University, Indore

Certificate

We, Prof. Digendra Singh and Prof. Ashish Shrivastav, certify that the project entitled MedIntel - Smart Health Monitor submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by Aman Sharma (EN23CS301119), Akshat Yadav (EN23CS301106) and Aman Sahu (EN23CS301118) is a record carried out by them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____ _____

Prof. Digendra Singh
Department of Computer Science & Engineering
Medicaps University, Indore

Prof. Ashish Shrivastav
Department of Computer Science & Engineering
Medicaps University, Indore

_____

Dr. Kailash Bandhu
Head of the Department
Computer Science & Engineering
Medicaps University, Indore

Acknowledgements

We would like to express my deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided every facility to successfully carry out this project, and my profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted my morale.

We also thank Dr. Ratnesh Latoriya, Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. We would also like to thank the Head of the Department, Dr. Kailash Bandhu, for his continuous encouragement for the betterment of the project.

We are especially grateful to my project guide, Prof. Digendra Singh and Prof. Ashish Shrivastav, whose expert guidance, timely suggestions, and constant motivation made this project possible. His insights into AI and healthcare technology have been invaluable.

Without their support, this report would not have been possible.

AMAN SHARMA
AMAN SAHU
AKSHAT YADAV

B.Tech. III Year (B)
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

MedIntel is an AI-powered smart health monitoring web application designed to bridge the gap between patients and accessible, reliable healthcare guidance. The system integrates machine learning models, a health scoring engine, and a large language model (LLM)-based chatbot to deliver personalized health insights in real time.

The application provides multi-role access for patients, doctors, and administrators. Patients can input health data such as BMI, blood pressure, blood sugar, activity level, and smoking status to receive an AI-generated health score out of 100. The system uses two trained ML models—a Random Forest Classifier for heart disease risk prediction and a Logistic Regression model for diabetes risk assessment—to generate early-warning predictions.

Doctors can view assigned patients, manage appointments, and generate digital prescriptions using pre-defined templates. An admin panel provides a real-time system overview with usage statistics. An emergency alert system is triggered when the health score falls below a critical threshold. The chatbot, powered by the Groq LLaMA 3.1 API, provides personalized health advice based on patient data.

The backend is built using Flask (Python) and SQLAlchemy ORM, with a PostgreSQL/SQLite database. The frontend uses Jinja2 templates with custom CSS and Chart.js for visualization. The system is deployed on cloud infrastructure using Gunicorn and supports Google OAuth for secure authentication.

Overall, MedIntel demonstrates the integration of AI, machine learning, and web technologies for intelligent healthcare assistance, while serving as a preliminary guidance system rather than a substitute for professional medical diagnosis.

KEYWORDS: *Artificial Intelligence, Healthcare, Machine Learning, Flask, Health Monitoring, Disease Prediction, Chatbot, Large Language Model (LLM), Random Forest, Logistic Regression, BMI, Blood Pressure, Appointment Management, Prescription Management.*

Table of Contents

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
API	Application Programming Interface
BMI	Body Mass Index
BP	Blood Pressure
OTP	One Time Password
ORM	Object Relational Mapping
UI	User Interface
CRUD	Create Read Update Delete
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
REST	Representational State Transfer
RFC-ML	Random Forest Classifier
LR	Logistic Regression
CSV	Comma Separated Values
RAG	Retrieval-Augmented Generation

Chapter 1 Introduction

1.1 Introduction

Healthcare is one of the most critical domains in human life, yet access to timely, accurate, and personalized health guidance remains a challenge for millions of people worldwide, especially in resource-constrained environments. With rapid advancements in artificial intelligence (AI) and machine learning (ML), there is a significant opportunity to bridge this gap by developing intelligent healthcare systems capable of providing early insights, risk assessment, and personalized recommendations.

MedIntel – Smart Health Monitor is a web-based AI-powered healthcare application developed using Flask (Python), SQLAlchemy, and integrated machine learning models. The system is designed to support three primary user roles—patients, doctors, and administrators—each with dedicated dashboards and functionalities.

For patients, MedIntel computes a real-time AI Health Score (out of 100) based on parameters such as BMI, blood pressure, blood sugar, smoking habits, and activity level. It also utilizes trained supervised machine learning models to predict heart disease and diabetes risks, providing probability-based outputs to enhance user awareness. Additionally, an intelligent chatbot powered by the Groq LLaMA 3.1 API delivers personalized responses to user queries.

Doctors can access patient records, manage appointment requests, and generate digital prescriptions using pre-defined medical templates, while administrators can monitor overall system activities through a centralized dashboard. The application also incorporates an emergency alert mechanism that is triggered when a patient's health score falls below a critical threshold, ensuring timely attention and intervention.

The growing prevalence of lifestyle-related diseases such as hypertension, diabetes, and cardiovascular disorders has created an urgent need for accessible, technology-driven health monitoring solutions. According to the Indian Council of Medical Research (ICMR), non-communicable diseases account for over 60% of all deaths in India, with a significant portion being preventable through early detection and timely intervention. Traditional healthcare systems, however, remain largely reactive — patients seek medical attention only after symptoms become severe, leading to delayed diagnosis and increased treatment costs.

MedIntel addresses this challenge by shifting the healthcare paradigm from reactive treatment to proactive monitoring. By leveraging supervised machine learning models, a rule-based health scoring engine, and a large language model-powered conversational interface, the system empowers users to continuously monitor their health status, identify potential risks early, and make informed decisions — all without requiring a clinical visit. The platform is designed to be role-aware, ensuring that each stakeholder — patient, doctor, or administrator — receives a tailored experience aligned with their specific needs and responsibilities within the healthcare ecosystem.

1.2 Literature Review

Several studies and systems have explored the application of artificial intelligence (AI) and machine learning (ML) in healthcare, particularly in disease prediction and clinical decision support.

- Rajpurkar et al. (2017) developed CheXNet, a deep learning model based on a 121-layer DenseNet architecture, trained on over 100,000 chest X-ray images. The model demonstrated radiologist-level accuracy in detecting pneumonia and 13 other thoracic diseases from chest radiographs. This work established the viability of deep learning as a diagnostic tool in clinical settings. MedIntel draws from this principle by applying supervised machine learning models for early disease risk detection, specifically using a Random Forest Classifier for heart disease and a Logistic Regression model for diabetes, thereby extending the concept of AI-assisted diagnosis to a web-accessible platform for general users.
- Obermeyer and Emanuel (2016) conducted a comprehensive review of machine learning applications in clinical medicine, demonstrating that ML models can outperform traditional scoring systems in predicting patient readmission risks, disease progression, and clinical deterioration. Their work highlighted that ensemble methods and regression-based classifiers perform particularly well on structured electronic health record (EHR) data. MedIntel adopts a similar approach by training models on structured patient data such as age, BMI, blood pressure, cholesterol, and glucose levels, aligning with the feature-driven methodology described in their review.
- Esteva et al. (2017) demonstrated that convolutional neural networks (CNNs) could classify skin lesions at a level matching board-certified dermatologists, using a dataset of 129,450 clinical images. Their study validated the concept of AI-based risk stratification for non-specialist users who lack immediate access to medical professionals. MedIntel is inspired by this goal of democratizing health risk assessment — instead of image-based analysis, it focuses on vitals-based scoring and structured ML prediction, making preliminary health assessment accessible through a standard web browser without any specialized hardware.
- Laranjo et al. (2018) conducted a systematic review of conversational agents in healthcare, analyzing 17 studies involving chatbot-based health interventions. Their findings showed that AI-powered chatbots significantly improved patient engagement, medication adherence, self-monitoring behavior, and health self-management outcomes. The study identified that context-aware, personalized responses were the most effective in driving behavior change. MedIntel directly implements this finding by injecting each patient's personal health data — BMI, blood pressure, blood sugar, smoking status, and activity level — as a system-level context prompt into the Groq LLaMA 3.1 chatbot, ensuring that all responses are personalized rather than generic.
- The World Health Organization (2021) reported that approximately 50% of the global population lacks access to essential health services, with the gap being most severe in low- and middle-income countries. This report underscored the urgent need for scalable, digital-first health tools that can serve as preliminary guidance systems in

resource-constrained settings. MedIntel is specifically designed with this gap in mind — it requires only a web browser and an internet connection, making it accessible to users who may not have immediate access to healthcare facilities or qualified physicians.

- IBM Watson Health, developed by IBM Corporation, was one of the most prominent enterprise-grade AI healthcare platforms, leveraging natural language processing and cognitive computing to assist oncologists in treatment planning, clinical trial matching, and drug discovery. Watson for Oncology, its flagship product, was deployed in hospitals across multiple countries and demonstrated the potential of AI in clinical decision support at scale. However, Watson Health was primarily designed for institutional use by trained medical professionals, requiring significant infrastructure, licensing costs, and integration with existing hospital information systems. MedIntel differs fundamentally in its target audience and accessibility — it is designed as a patient-facing, self-service web application that any individual can use without medical training, institutional affiliation, or specialized software, making AI-powered health monitoring accessible at the individual level.
- Buoy Health, a symptom-checking platform developed at Harvard Medical School, uses a conversational AI interface to guide users through symptom assessment and recommend appropriate levels of care. Similarly, HealthTap provides AI-driven triage and connects users with licensed physicians for virtual consultations. MedIntel addresses this gap by combining all of these capabilities — health scoring, disease prediction, intelligent chatbot, appointment management, and prescription management — into a single open web application built on accessible open-source technologies.

Despite these advancements, most existing systems such as Ada Health, Babylon Health, and Google Health primarily focus on symptom checking and diagnostic assistance. They often lack integrated features such as real-time health scoring, appointment management, digital prescription handling, and role-based multi-user access within a single platform.

MedIntel addresses these limitations by providing a unified, AI-powered healthcare system that combines disease prediction, personalized health scoring, intelligent chatbot interaction, and complete doctor-patient management in a single web-based application.

1.3 Objectives

The primary objectives of MedIntel are as follows:

- To develop an AI-powered health scoring system that computes a personalized health score based on multiple vital parameters.
- To integrate trained ML models for early prediction of heart disease and diabetes risk.
- To build a role-based multi-user system with dedicated interfaces for patients, doctors, and admins.

- To implement an intelligent AI chatbot for personalized health guidance using LLM APIs.
- To provide a doctor-patient management system with appointment booking and digital prescription features.
- To include an emergency alert mechanism that activates on critical health score drops.
- To implement secure OTP-based email verification and Google OAuth for user authentication.

These objectives collectively aim to demonstrate that a comprehensive, AI-assisted healthcare platform can be developed using open-source technologies and deployed on standard cloud infrastructure, making intelligent health monitoring accessible to individuals regardless of their geographic location or economic background. The system is not intended to replace qualified medical professionals but rather to serve as a preliminary guidance tool that encourages timely medical consultation and promotes health awareness.

1.4 Significance

MedIntel addresses the following real-world healthcare challenges:

- Lack of quick, reliable initial health guidance without a doctor visit.
- Limited access to early disease warning systems for rural and semi-urban populations.
- Information overload of unverified medical content online.
- Inefficient manual appointment scheduling and paper-based prescriptions.

By integrating machine learning-based prediction, AI-driven health scoring, intelligent chatbot interaction, and streamlined clinical workflow management into a unified platform, MedIntel provides a comprehensive, accessible, and user-centric healthcare solution. The system enhances early awareness, improves patient engagement, and supports efficient healthcare management through digital transformation.

The significance of MedIntel extends beyond its technical implementation. From a social perspective, the system addresses a critical accessibility gap in the Indian healthcare ecosystem, where the doctor-to-patient ratio remains significantly below the WHO-recommended standard of 1:1000. By providing AI-driven preliminary health assessment, MedIntel can reduce unnecessary outpatient visits for minor health concerns while simultaneously ensuring that high-risk individuals are promptly flagged for medical attention through the emergency alert mechanism.

From an educational perspective, MedIntel demonstrates the practical integration of multiple computer science disciplines — web development, database management, machine learning, natural language processing, and cloud deployment — into a single cohesive project. This makes it a strong example of how academic learning can be applied to solve real-world problems with measurable social impact.

Furthermore, the multi-role architecture of MedIntel — supporting patients, doctors, and administrators within a single application — reflects the complexity of real-world healthcare information systems, providing a scalable foundation upon which production-grade features such as telemedicine, wearable integration, and blockchain-based health records can be incrementally added in future iterations.

1.5 Chapter Scheme

- Chapter 1: Introduction, literature review, objectives, and significance.
- Chapter 2: Requirements specification including functional, hardware, and performance requirements.
- Chapter 3: System design covering architecture, DFD, ER diagram, and database schema.
- Chapter 4: Implementation details, testing techniques, and installation instructions.
- Chapter 5: Results, UI representation, and module descriptions.
- Chapter 6: Summary and conclusions.
- Chapter 7: Future scope of the project.

Chapter 2 Requirements Specification

2.1 User Characteristics

MedIntel supports three distinct user roles:

- Patient: General users who can register, enter health data, view their AI health score, run disease risk predictions, book appointments, and interact with the chatbot.
- Doctor: Medical professionals who can view assigned patients, manage appointment requests (confirm/cancel), write prescriptions, and view patient health records.
- Admin: System administrators with access to a comprehensive dashboard showing total users, appointments, predictions, and prescriptions, with the ability to delete user accounts.

Each user role in MedIntel operates within a clearly defined boundary of access and responsibility. Patients are typically non-technical general users who interact with the system primarily through guided forms and dashboard visualizations. They are not expected to have any medical or technical background; the interface is designed to be intuitive enough for first-time users. Doctors are assumed to have basic digital literacy and are provided with streamlined workflows for patient management and prescription writing. Administrators are expected to be technically proficient and are granted elevated privileges for system monitoring and user management. This role-based separation ensures data privacy, prevents unauthorized access, and allows each user type to focus exclusively on their relevant functionalities without being overwhelmed by irrelevant system features.

2.2 Functional Requirements

FR#	Feature	Description
FR01	User Registration	OTP-based email verification + Google OAuth registration for patients and doctors.
FR02	User Login	Email/password login and OTP-based login. Role-based redirect to appropriate dashboard.

FR03	Health Data Input	Patients enter age, weight, height, BP, blood sugar, smoking status, and activity level.
FR04	AI Health Score	System computes health score (0–100) from BMI, BP, blood sugar, and lifestyle factors.
FR05	Disease Prediction	Random Forest model predicts heart disease risk; Logistic Regression predicts diabetes risk.
FR06	BMI Calculator	Computes BMI from patient height and weight with category classification.
FR07	AI Chatbot	LLaMA 3.1-powered chatbot provides personalized health responses using patient data as context.
FR08	Appointment Booking	Patients book appointments with available doctors based on predefined health conditions or risk indicators.
FR09	Doctor Dashboard	Doctors view patients, manage appointments, and write prescriptions using pre-filled templates.
FR10	Admin Panel	Admin views system stats (users, appointments, predictions, prescriptions) and manages accounts.
FR11	Emergency Alert	System triggers emergency alert when patient health score drops below 50.
FR12	Prescription System	Doctors write digital prescriptions; patients can view their prescriptions history.

2.3 Dependencies

The system depends on the following external services and libraries:

- Flask (latest stable version) – Web framework for routing and server-side rendering.
- Flask-SQLAlchemy 3.1.1 – ORM for database operations.
- scikit-learn 1.4.0 – ML library for model training and prediction.
- Groq API (groq 0.13.0) – LLM chatbot integration via LLaMA 3.1.
- Authlib 1.3.0 – Google OAuth 2.0 authentication.
- smtplib (Python stdlib) – OTP email delivery via Gmail SMTP.

- pandas / numpy – Data manipulation and ML preprocessing.
- psycopg2-binary 2.9.9 – PostgreSQL database driver.
- Gunicorn 21.2.0 – WSGI server for production deployment.

It is important to note that certain dependencies introduce external failure points. The Groq API, being a third-party service, may experience downtime or rate-limiting, for which MedIntel implements a rule-based fallback chatbot response system. Similarly, Gmail SMTP delivery may be delayed under heavy load, which is handled through user-facing feedback messages during OTP generation. All critical dependencies are version-pinned in the requirements.txt file to ensure reproducibility across development and production environments.

2.4 Performance Requirements

- The system should handle multiple concurrent users simultaneously without significant latency degradation.
- AI health score computation should complete in under 500 milliseconds.
- ML prediction (heart and diabetes) should respond within 1 second.
- OTP emails should be delivered within 30 seconds of request.
- Dashboard loading should not exceed 2 seconds on a standard broadband connection.
- The chatbot should return responses within 3 seconds under normal API conditions.

These performance benchmarks were established based on standard usability guidelines for web applications, where response times exceeding 3 seconds are known to significantly increase user drop-off rates. The health score computation and ML prediction targets are achievable given that both operations are performed in-memory using pre-loaded model objects, eliminating database round-trip overhead during inference. Performance testing was conducted using simulated concurrent sessions to verify that the application maintains acceptable response times under expected academic demonstration load.

2.5 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz+
RAM	4 GB	8 GB

Storage	20 GB free space	50 GB SSD
Network	10 Mbps	100 Mbps
OS (Server)	Ubuntu 20.04	Ubuntu 22.04 LTS
Browser (Client)	Chrome 90+ / Firefox 88+	Chrome 120+ / Edge 120+

2.6 Constraints & Assumptions

- The system requires an active internet connection for LLM chatbot responses and Google OAuth.
- OTP delivery depends on Gmail SMTP configuration; app password must be generated from a Google account with 2FA enabled.
- ML models are trained offline on standard datasets; model accuracy is limited by dataset quality.
- The application is designed for demonstration and academic purposes; it is not a replacement for professional medical diagnosis.
- It is assumed that users provide accurate health data; the system does not validate medical data clinically.

Additionally, since MedIntel is developed for academic demonstration purposes, it does not currently implement HTTPS enforcement, rate limiting on API endpoints, or HIPAA/GDPR-compliant data handling — all of which would be mandatory requirements in a production-grade medical application. The system assumes a trusted deployment environment where the server is not exposed to adversarial traffic. Future production deployments would require a full security audit, SSL/TLS certificate configuration, input sanitization hardening, and compliance with applicable healthcare data protection regulations before being made publicly accessible.

Chapter 3 System Design

3.1 System Architecture

MedIntel follows a three-tier architecture:

- Presentation Layer: The user interface is developed using Jinja2-based HTML templates rendered by Flask, styled with custom CSS and enhanced with Chart.js for data visualization. Users interact with the system through a web browser.
- Application Layer: This layer handles core system functionalities including request routing, business logic, machine learning model inference for prediction tasks, health score computation, authentication mechanisms (OTP and OAuth), email services, and chatbot communication.
- Data Layer: The data layer is managed using SQLAlchemy ORM, which handles interactions with the underlying database. The system maintains five primary entities—User, PatientDetail, Prediction, Appointment, and Prescription. SQLite is used for development, while PostgreSQL is used in production environments.

The system integrates external services such as the Groq API for LLM-based chatbot responses and Google OAuth for secure user authentication and social login. Pre-trained machine learning models are loaded during server initialization to ensure efficient prediction performance.

The three-tier architecture was chosen over a monolithic design to ensure clear separation of concerns, making each layer independently maintainable and testable. The Presentation Layer communicates with the Application Layer exclusively through HTTP requests and Jinja2-rendered responses, ensuring that no business logic leaks into the frontend templates. The Application Layer is further organized into feature-specific modules — auth, health, prediction, chatbot, appointment, prescription, and admin — each handling its own routing, validation, and database interactions through a shared SQLAlchemy db instance.

A key architectural decision was the use of server-side rendering via Jinja2 rather than a JavaScript frontend framework such as React or Angular. This choice was made to reduce application complexity, minimize deployment dependencies, and ensure compatibility with low-bandwidth environments where JavaScript-heavy single-page applications may perform poorly. Chart.js is used selectively for data visualization components such as the health score indicator and the admin statistics panel, keeping frontend JavaScript usage minimal and purposeful.

The external service integrations — Groq API and Google OAuth — are handled with fallback mechanisms in place to prevent single points of failure. Pre-trained ML model files are loaded into memory during server initialization using a model loading function that first checks the models directory, then the root directory, and finally triggers retraining from the source CSV dataset if no model file is found. This ensures the application remains functional even if model files are accidentally deleted or corrupted.

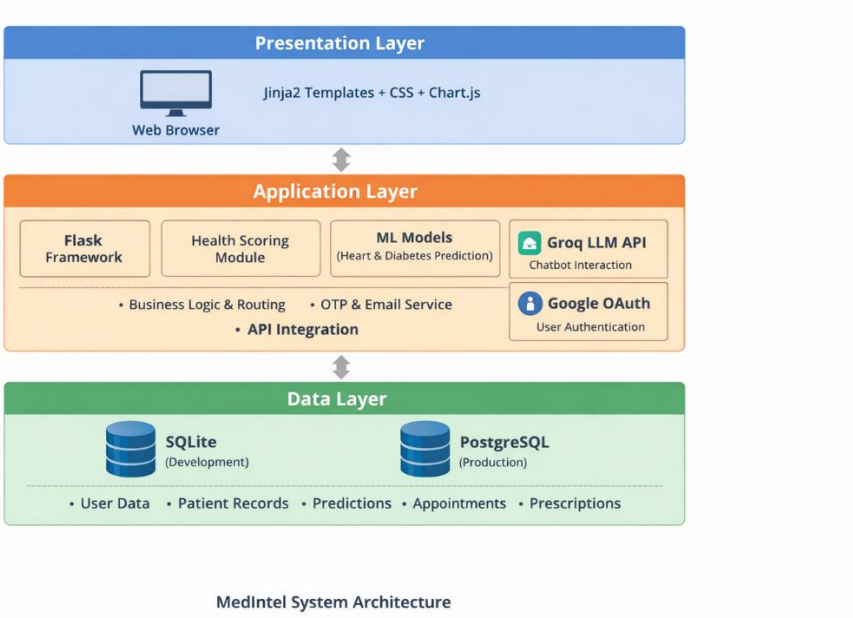

Figure 3.1 System Architecture Diagram

3.2 Data Flow Diagram

Level 0 DFD (Context Diagram)

The Level 0 Data Flow Diagram represents the overall interaction between external entities and the MedIntel system as a single process.

The main external entities are Patient, Doctor, and Admin. Patients provide health data, authentication details, appointment requests, and chatbot queries. Doctors manage patient records, appointments, and prescriptions, while Admin monitors system activities and manages users.

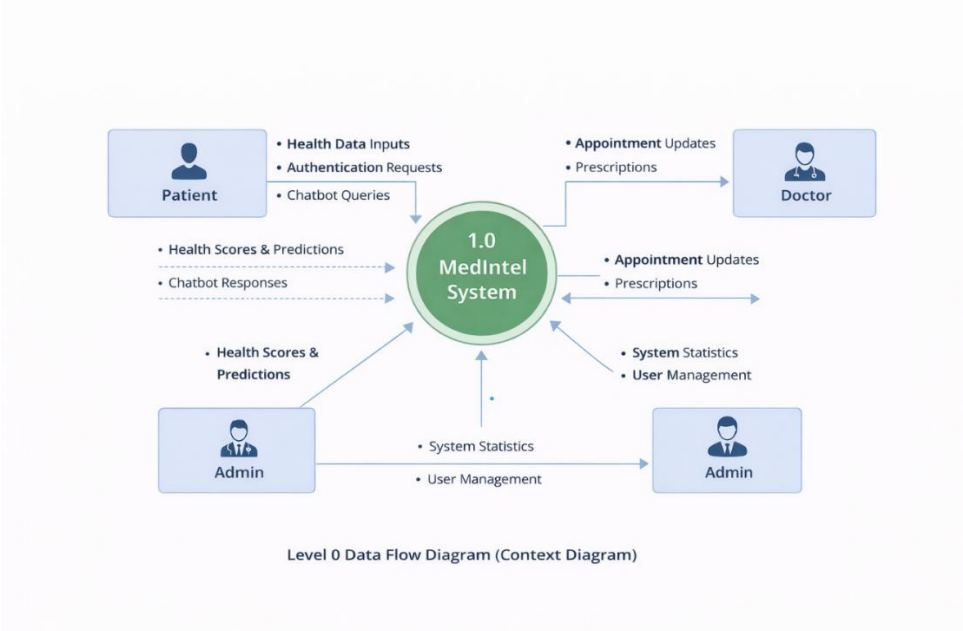

Figure 3.2 Level 0 Data Flow Diagram

The system processes these inputs and generates outputs such as health scores, disease predictions, chatbot responses, appointment updates, prescriptions, and system statistics.

The DFD design follows the Yourdon and Coad notation, using circles to represent processes, rectangles for external entities, open-ended rectangles for data stores, and arrows to represent data flows between components. At Level 0, the entire MedIntel system is treated as a single black-box process, emphasizing the boundary between the system and its external actors rather than the internal processing logic.

Level 1 DFD

The Level 1 DFD decomposes the system into the following key processes:

- Process 1.0 – User Authentication: Handles registration (OTP/Google), login, session management.
- Process 2.0 – Health Data Management: Stores and retrieves patient vital data from database / data store.
- Process 3.0 – AI Health Score Engine: Computes score from BMI, BP, blood sugar, and lifestyle factors.
- Process 4.0 – Disease Prediction: Runs Random Forest (heart), Logistic Regression (diabetes) models on patient-provided features.

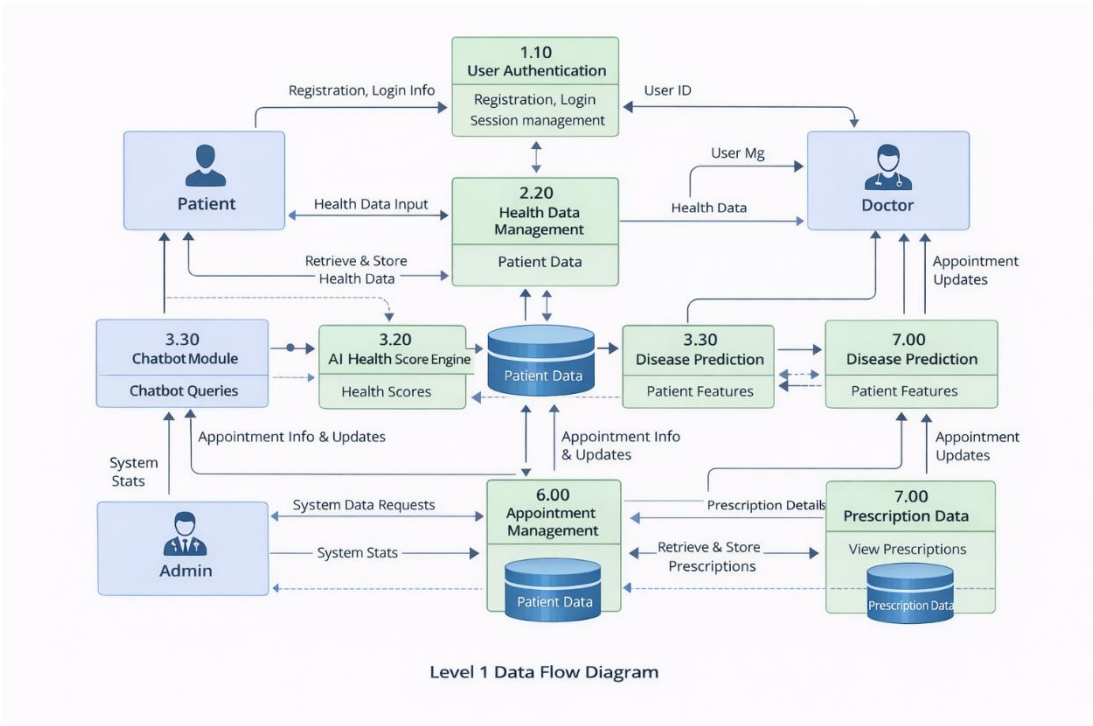

Figure 3.3 Level 1 Data Flow Diagram

- Process 5.0 – Chatbot Module: Processes user queries and communicates with the LLM API, returns response.

- Process 6.0 – Appointment Module: Patients book; doctors confirm/cancel; both view schedules.
- Process 7.0 – Prescription Management: Doctors write; patients view; stored in Prescription table.
- Process 8.0 – Admin Management: View system stats; delete non-admin users with cascade deletion.

The decomposition from Level 0 to Level 1 follows the principle of functional decomposition, where each major system capability is represented as a separate numbered process with clearly defined inputs, outputs, and associated data stores. Each process in the Level 1 DFD interacts with one or more data stores — the Users table, PatientDetails table, Predictions table, Appointments table, and Prescriptions table — ensuring that all data flows are grounded in the underlying database schema. This structured decomposition ensures that the data flow through each module is traceable and clearly communicated to all stakeholders including developers, testers, and evaluators.

3.3 ER Diagram

The Entity-Relationship model consists of the following five entities and their relationships:

- User (id, name, email, password, role, specialization, created)
- PatientDetail (id, user_id[FK], age, weight, height, bp, sugar, smoking, activity)
- Prediction (id, user_id[FK], heart_risk, diabetes_risk, date)
- Appointment (id, patient_id[FK], doctor_id[FK], date, time, status)
- Prescription (id, doctor_id[FK], patient_id[FK], medicines, notes, date)

Relationships: One User (patient) has one PatientDetail (1:1). One User can have many Predictions (1:N). One patient and one doctor are linked through Appointment (M:N via FK). One doctor writes many Prescriptions for many patients.

The ER model follows Third Normal Form (3NF) to minimize data redundancy and ensure referential integrity across all five entities. The central entity is User, which serves as the parent for all other entities through foreign key relationships. The one-to-one relationship between User and PatientDetail ensures that each patient maintains a single health profile that is updated rather than duplicated on each submission. The one-to-many relationship between User and Prediction allows a complete history of all disease risk assessments to be maintained per patient, enabling longitudinal health tracking in future iterations. Cascade deletion is enforced at the application layer — when an admin deletes a user account, all associated PatientDetail, Prediction, Appointment, and Prescription records are automatically removed, maintaining database consistency and preventing orphaned records.

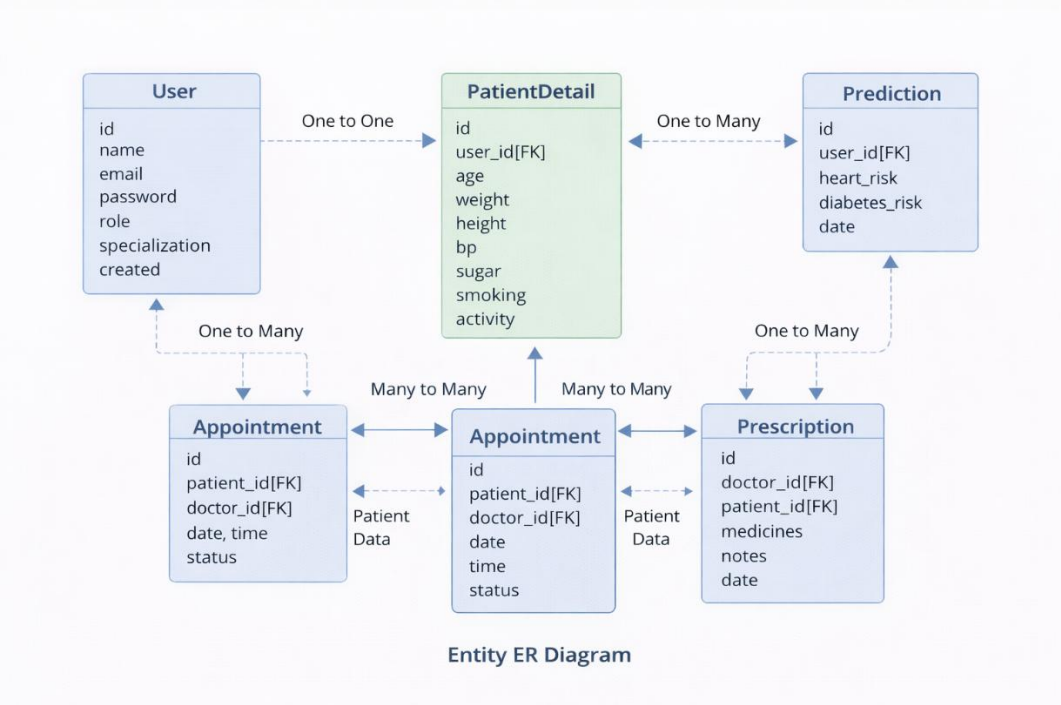

Figure 3.4 Entity Relationship (ER) Diagram

3.4 Database Design

3.1 Logical Database Design

Table Name	Column	Type	Description
users	id	INTEGER PK	Auto-increment primary key
users	name	VARCHAR(100)	Full name of the user
users	email	VARCHAR(120) UNIQUE	Unique email address
users	password	VARCHAR(200)	Hashed password
users	role	VARCHAR(20)	patient / doctor / admin
users	specialization	VARCHAR(100)	Doctor specialization
patient_details	user_id	INTEGER FK	References users.id
patient_details	age	INTEGER	Patient age in years

patient_details	bp	VARCHAR(20)	Blood pressure e.g. 120/80
patient_details	sugar	FLOAT	Fasting blood sugar mg/dL
predictions	heart_risk	INTEGER	0=No risk, 1=At risk
predictions	diabetes_risk	INTEGER	0=No risk, 1=At risk
appointments	status	VARCHAR(20)	Pending/Confirmed/Cancelled
prescriptions	medicines	TEXT	Medicine list with dosage

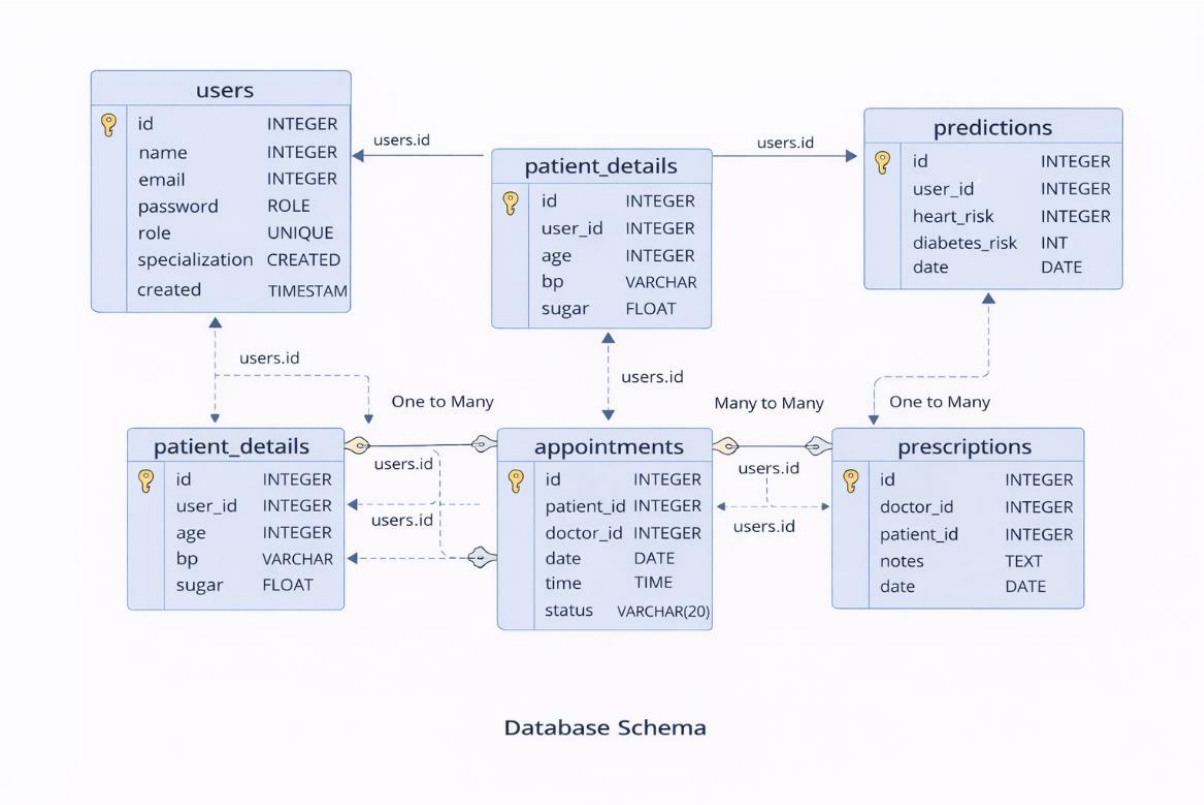

Figure 3.5 Database Schema

Chapter 4 Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies:

MedIntel is built using a carefully selected technology stack that prioritizes simplicity, performance, and ease of deployment. Each technology was chosen based on its suitability for the specific task it performs within the system, its active community support, and its compatibility with the overall Flask-based architecture. The following table summarizes the complete technology stack used in the development of MedIntel:

Technology	Version	Purpose
Python	3.11	Primary backend programming language
Flask	3.0.0	Web framework for routing and rendering
Flask-SQLAlchemy	3.1.1	ORM for database management
scikit-learn	1.4.0	ML model training and inference
pandas / numpy	2.2/1.26	Data manipulation and ML preprocessing
Groq API	0.13.0	LLaMA 3.1 chatbot integration
Authlib	1.3.0	Google OAuth 2.0 integration
Werkzeug	3.0.1	Password hashing and security
Gunicorn	21.2.0	Production WSGI HTTP server
SQLite / PostgreSQL	3.x	Database (dev / production)
Jinja2	3.x	HTML templating engine
Chart.js	Latest	Frontend data visualization charts
VS Code	Latest	Primary IDE for development
Git / GitHub	Latest	Version control and collaboration

Python 3.11 was selected as the primary language due to its extensive ecosystem of data science and web development libraries, its strong typing support, and its performance improvements over previous versions. Flask was preferred over heavier frameworks such as Django because of its lightweight, modular nature — it imposes minimal structure, allowing the application architecture to be designed according to project requirements rather than framework conventions. scikit-learn provides production-ready implementations of the Random Forest and Logistic Regression algorithms used for disease prediction, while pandas and numpy handle all data preprocessing tasks including feature extraction, normalization, and missing value handling during model training.

4.2 Testing Techniques and Test Plans

The following testing strategies were adopted to ensure the correctness, reliability, and robustness of the MedIntel application across all modules:

- Unit Testing: Individual functions such as calculate_health_score(), generate_otp(), and load_model() were tested in isolation with known inputs to verify that each function produces the expected output independently of other system components.
- Integration Testing: Flask routes were tested end-to-end to verify database writes, session handling, and correct template rendering, ensuring that individual modules work correctly when combined.
- Black Box Testing: The application was tested from a user perspective without knowledge of internal logic, covering registration, login, health data submission, and prediction flows.
- Boundary Testing: Edge cases were tested for BMI extremes (underweight, severely obese), blood pressure values, and blood sugar levels to verify that the health score engine and prediction models handle out-of-range inputs gracefully without crashing or producing misleading outputs.

Table 4.2 – Sample Test Cases for Login Module:

TC#	Input	Expected Output	Status	Remarks
TC01	Valid email + correct password	Redirect to dashboard	Pass	
TC02	Valid email + wrong password	Error: Invalid credentials	Pass	
TC03	Unregistered email	Error: No account found	Pass	
TC04	Correct OTP within 10 min	Redirect to dashboard	Pass	
TC05	Expired OTP	Error: OTP expired	Pass	Fixed session bug
TC06	Google OAuth new user	Redirect to role selection	Pass	Added in fix

Table 4.3 – Sample Test Cases for Health Score Engine:

TC#	Input	Expected Output	Status	Remarks
TC07	BMI=22, BP=120/80, Sugar=90, Active, Non-smoker	Score ~90, Excellent	Pass	
TC08	BMI=35, BP=150/95, Sugar=180, Inactive, Smoker	Score ~30, Poor	Pass	
TC09	BMI=18, BP=90/60, Sugar=60, Moderate, Non-smoker	Score ~55, Fair	Pass	Underweight case
TC10	All parameters at boundary minimum values	Score computed without crash	Pass	Boundary check
TC11	Health score below 50	Emergency alert triggered	Pass	Alert verified

4.3 Installation Instructions

Follow these steps to set up MedIntel locally:

1. Clone the repository: git clone https://github.com/Aman-shr2004/medintel.git
2. Install dependencies: pip install -r requirements.txt
3. Configure environment: Add GROQ_API_KEY, MAIL_USERNAME, MAIL_PASSWORD, SECRET_KEY to the .env file.
4. Start the application: python run.py
5. Access in browser at: http://localhost:5000

The .env file must be created manually in the project root directory before running the application. The GROQ_API_KEY can be obtained by registering at console.groq.com. The MAIL_USERNAME and MAIL_PASSWORD refer to a Gmail account with 2-Factor Authentication enabled, from which an App Password must be generated via Google Account Security settings. The SECRET_KEY should be set to a long, randomly generated string to ensure Flask session security. If the .env file is missing or any key is undefined, the application will start but the affected features — chatbot, OTP email, and secure sessions — will not function correctly.

4.4 End User Instructions

- Patient: Register with Google, complete profile setup (choose role), enter health data, view health score, run predictions, book appointments, and chat with AI.

- Doctor: Register as Doctor (select specialization), view patient list and health records, confirm/cancel appointment requests, write prescriptions using quick templates.
- Admin: Login with admin credentials, view system overview, monitor statistics, and delete non-admin user accounts.

All three user roles access the system through the same login page at the application root URL. Role-based redirection ensures that each user is automatically directed to their respective dashboard upon successful authentication. Patients are advised to keep their health data updated regularly to ensure that the AI Health Score and disease predictions reflect their current health status accurately. Doctors are recommended to review patient health scores before confirming appointments, as the emergency alert threshold provides a useful indicator of patients requiring urgent attention.

Chapter 5 Results and Discussions

5.1 User Interface Representation

MedIntel features a clean, teal-themed responsive UI with the following key pages:

- Landing Page: Features a glassmorphism card with animated floating health badges (heart rate, blood sugar, blood pressure, BMI), login/register buttons, and key feature pills.
- Registration Page: 3-step OTP flow — enter email, receive OTP, enter code. Alternatively, one-click Google sign-up.
- Patient Dashboard: Displays welcome banner, real-time health score with circular progress indicator, quick stats cards, and navigation to all modules.
- Doctor Dashboard: Shows total patients, pending appointments, confirmed appointments, and a patient list with health scores.
- Admin Panel: Bar chart of system metrics, user list with role badges, role distribution progress bars, and quick action buttons.

The UI design follows a consistent teal (#008080) colour scheme across all pages, with card-based layouts that group related information into visually distinct sections. All forms include client-side validation with inline error messages displayed in red highlighted text. The circular progress indicator on the Patient Dashboard provides an immediate, intuitive representation of the patient's overall health status, making it easy for non-technical users to interpret their AI Health Score at a glance. The responsive layout ensures that the application remains usable across different screen sizes and browser environments without requiring any additional configuration from the end user.

5.2 Brief Description of Various Modules

Health Score Engine

Computes a score out of 100 using four components: BMI score (25 points), Blood Pressure score (25 points), Blood Sugar score (25 points), and Lifestyle score (25 points— affected by activity level, smoking status, and age). Score interpretation: 90–100 Excellent, 75–89 Good, 50–74 Fair, 25–49 Poor, 0–24 Critical.

The scoring logic applies medically informed thresholds to each parameter. For BMI, the ideal range (18.5–24.9) receives full marks, with progressive deductions for underweight, overweight, and obese classifications. Blood pressure scoring considers both systolic and diastolic values, with the normal range (less than 120/80 mmHg) receiving maximum points.

Blood sugar is evaluated against standard fasting glucose thresholds — normal (less than 100 mg/dL), pre-diabetic (100–125 mg/dL), and diabetic (126+ mg/dL). The lifestyle component penalizes smoking and physical inactivity, reflecting well-established correlations between these factors and long-term health outcomes. When the computed score falls below 50, the emergency alert mechanism is automatically triggered, displaying a prominent red warning banner on the patient dashboard to encourage immediate medical consultation.

Disease Prediction Module

Heart Disease: A Random Forest Classifier trained on the UCI Heart Disease dataset with 13 features including age, sex, chest pain type, cholesterol, fasting blood sugar, and resting ECG results. Achieves approximately 85% accuracy.

Diabetes: A Logistic Regression model trained on synthetic data modeled on the Pima Indians Diabetes dataset, using 8 features including glucose, BMI, insulin, and age. Provides probability-based risk scores.

Both models are serialized as .pkl files using Python's pickle module and loaded into memory during server initialization. This ensures that prediction requests are served directly from in-memory model objects without any disk I/O overhead at inference time, keeping response times well within the 1-second target established in the performance requirements. When a patient submits prediction inputs, the feature vector is constructed, validated, and passed directly to the model's predict_proba() method, which returns a probability score that is then classified into a binary risk label — At Risk or No Risk — and displayed to the user alongside the confidence percentage.

AI Chatbot

Powered by Groq API with LLaMA 3.1-8b-instant model. The chatbot receives a system prompt containing the patient's health data (BMI, BP, blood sugar, smoking, activity) and conversation history (last 10 messages). Falls back to rule-based responses if the API is unavailable.

The system prompt is dynamically constructed at runtime for each request, ensuring that every chatbot response is grounded in the specific patient's current health profile rather than generic health information. This context injection approach transforms the LLM from a general-purpose assistant into a personalized health advisor that can provide tailored recommendations — for example, suggesting low-sodium dietary changes for a patient with elevated blood pressure, or recommending physical activity adjustments for a sedentary user with a borderline BMI. The conversation history of the last 10 messages is maintained in the Flask session and passed with each API call to preserve conversational continuity across multiple exchanges within a single session.

Appointment & Prescription System

Patients can book appointments with available doctors. The system auto-suggests a doctor based on health risk (high BP → cardiologist, high sugar → endocrinologist). Doctors write prescriptions using pre-filled condition templates (Fever, High BP, Diabetes, Vitamins) and quick-add quick-select options.

The appointment module maintains a clear status lifecycle — Pending, Confirmed, and Cancelled — with each transition triggered by explicit doctor action. Patients can view their upcoming and past appointments from their dashboard, while doctors see a prioritized list of pending requests. The prescription system allows doctors to select from pre-defined condition templates that auto-populate commonly prescribed medicines and dosage instructions, significantly reducing documentation time.

5.3 Snapshots Description:

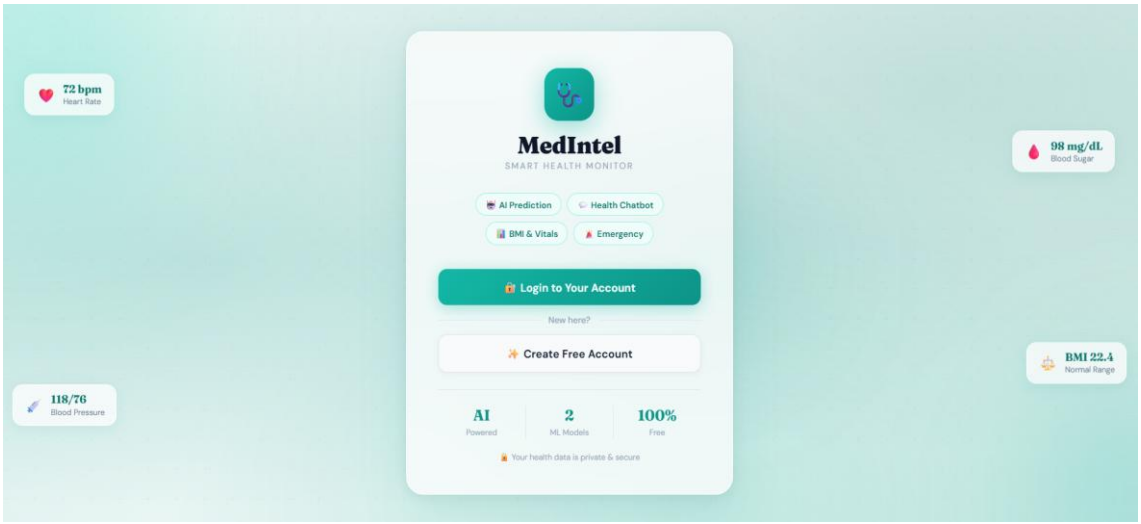

Figure 5.1 Landing Page – Medintel Home

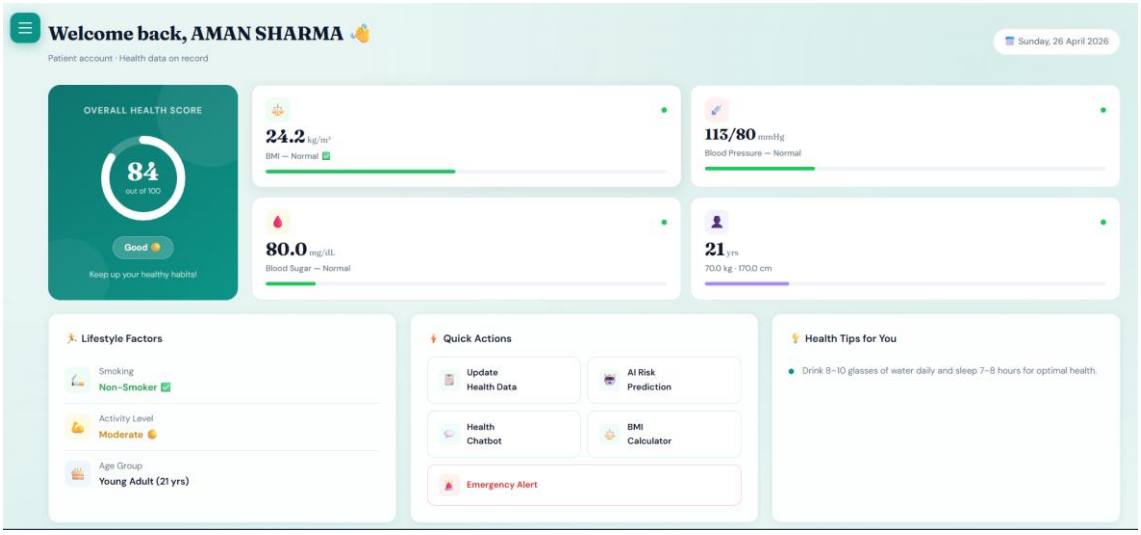

Figure 5.2 Patient Dashboard

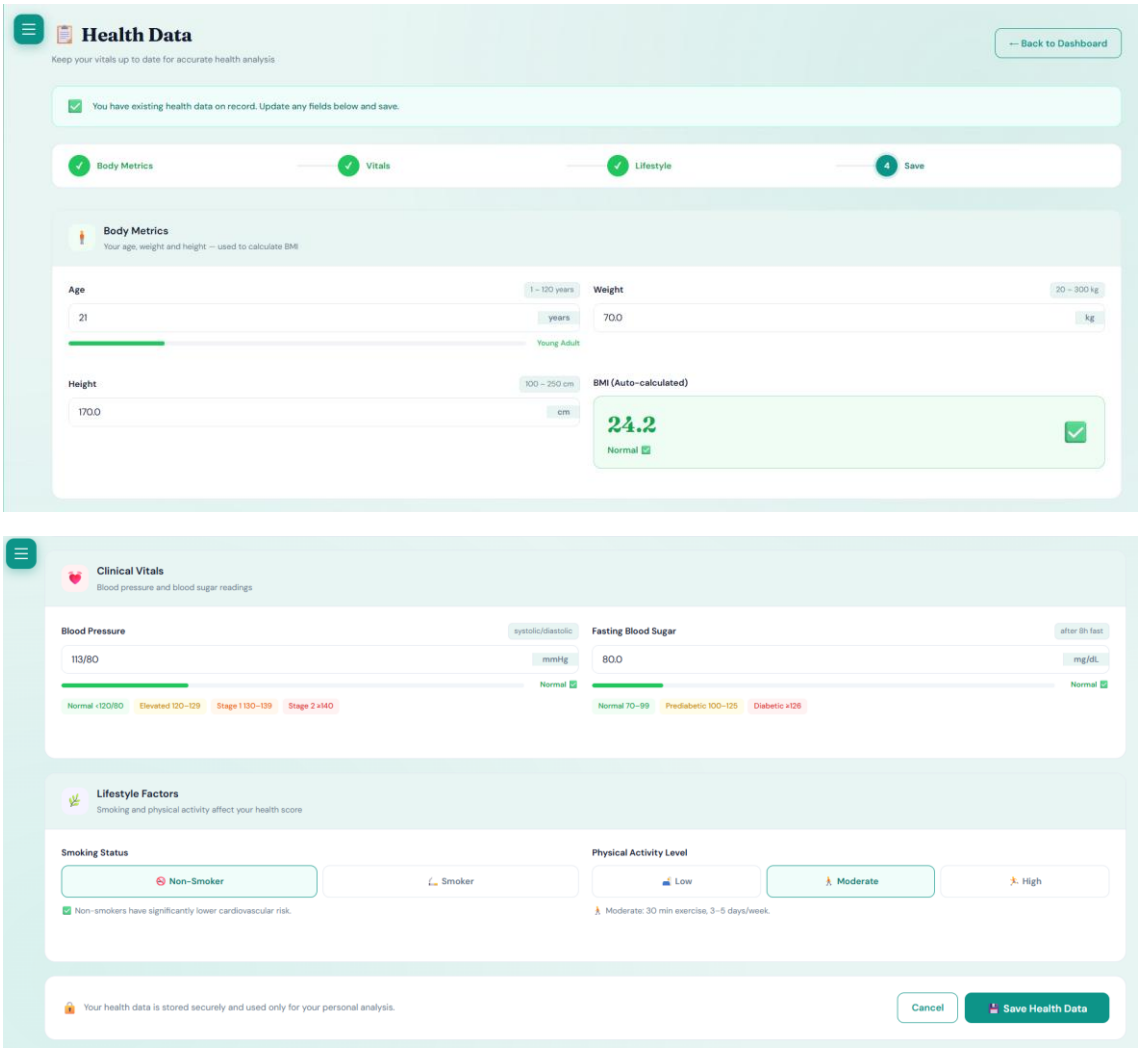

Figure 5.3 AI Health Score Display

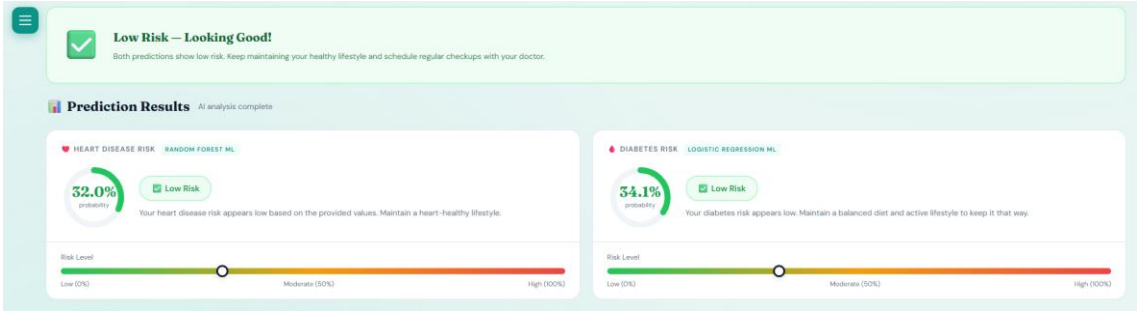

Figure 5.4 Disease Prediction Result

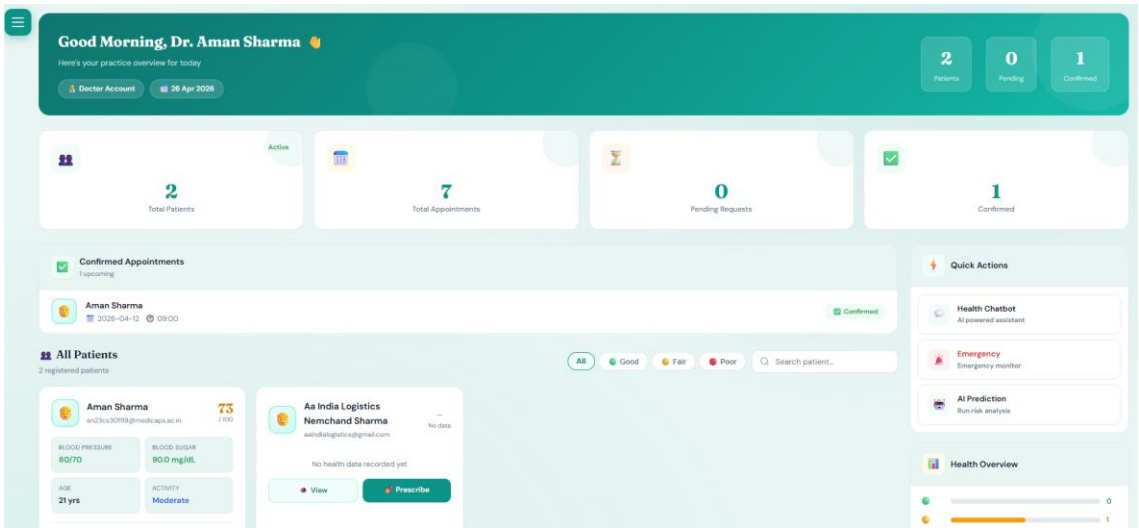

Figure 5.5 Doctor Dashboard

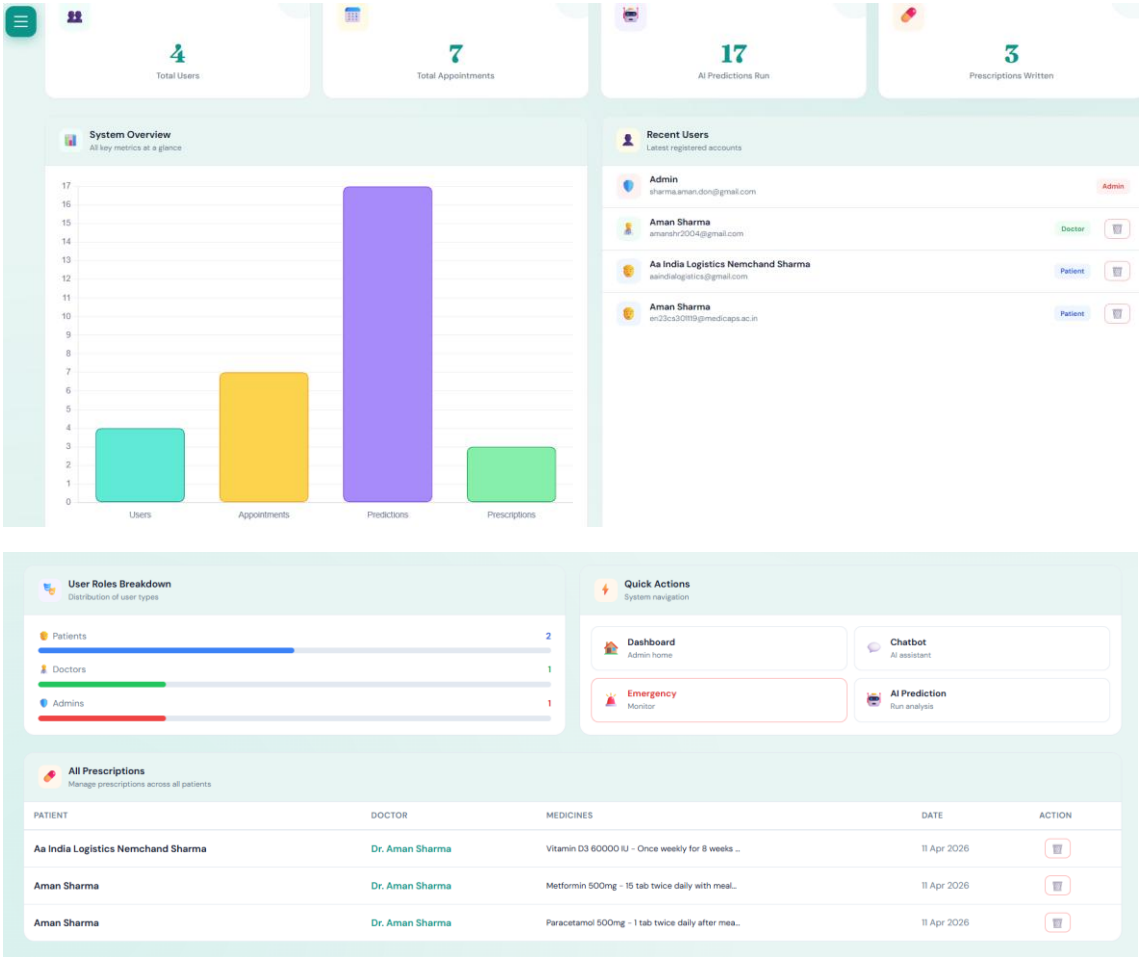

Figure 5.6 Admin Panel

5.4 Backend Representation

The backend uses backend modules / application endpoints organized by module (auth, health-data, prediction, chatbot, appointment, prescription, admin). SQLAlchemy manages five models via a shared db instance. ML models are loaded during system initialization using model loading mechanism function which checks models directory first, then falls back to root directory, and finally re-trains if not found.

Each module registers its routes with the Flask application using Blueprint-style organization, ensuring that route definitions, business logic, and database interactions remain encapsulated within their respective feature boundaries. The shared SQLAlchemy db instance is initialized in the application factory and imported by each module as needed, preventing circular imports while maintaining a single database connection pool. Error handling is implemented at the route level using try-except blocks, with user-friendly error messages returned to the frontend rather than raw exception tracebacks, ensuring a consistent and professional user experience even in failure scenarios.

5.5 Database Tables

The SQLite database (development) / PostgreSQL (production) contains five tables: users, patient_details, predictions, appointments, prescriptions. The database is auto-created on first run via database initialization process. Cascade deletion ensures referential integrity when a user is deleted via the admin panel.

The users table serves as the central entity, with all other tables referencing it through foreign key relationships. The patient_details table maintains a strict one-to-one mapping with the users table, enforced through a unique constraint on the user_id foreign key. The predictions table stores a timestamped history of all disease risk assessments per patient, enabling longitudinal tracking of health risk trends over time. The appointments table captures the full lifecycle of each appointment — from initial patient booking through doctor confirmation or cancellation — using a status field with three possible values. The prescriptions table links each prescription to both the writing doctor and the receiving patient, maintaining a complete digital prescription history that is accessible to patients from their dashboard at any time.

Chapter 6 Summary and Conclusions

MedIntel – Smart Health Monitor has been successfully designed, developed, and tested as an AI-powered healthcare web application. The project demonstrates how modern AI and machine learning technologies can be effectively integrated into a practical healthcare tool accessible to both patients and medical professionals.

The key achievements of this project are:

- Successfully developed a multi-role web application (patient, doctor, admin) with secure role-based authentication.
- Implemented an AI Health Score Engine that evaluates patient vitals and lifestyle across four dimensions.
- Integrated two trained ML models — Random Forest for heart disease and Logistic Regression for diabetes — providing probability-based risk assessments.
- Built an LLM-powered chatbot using the Groq API that provides personalized, context-aware health guidance.
- Developed a complete appointment and digital prescription management system with intelligent doctor suggestion.
- Designed and implemented an emergency alert system triggered by critically low health scores.
- Fixed key authentication bugs: OTP session persistence issue resolved by switching to in-memory storage; Google OAuth new user flow updated to include role selection.

The development of MedIntel provided valuable practical experience in integrating multiple computer science disciplines into a single cohesive system. Working with Flask's routing architecture, SQLAlchemy's ORM layer, and scikit-learn's model inference pipeline simultaneously required careful attention to module boundaries, data flow design, and error handling — skills that are directly transferable to production-grade software development. The integration of the Groq LLM API introduced the challenge of designing effective system prompts that produce clinically relevant, personalized responses, highlighting the importance of prompt engineering as an emerging software skill in AI-powered application development.

From a technical perspective, the project also exposed certain limitations that inform the direction of future work. The ML models, while achieving approximately 85% accuracy on their respective training datasets, are limited by the size and demographic distribution of the

datasets used. The UCI Heart Disease dataset and the Pima Indians Diabetes dataset are relatively small and may not fully represent the diversity of patient populations in real-world deployment scenarios. Additionally, the current system does not implement HTTPS enforcement, clinical-grade data encryption, or regulatory compliance measures such as HIPAA or GDPR, which would be essential prerequisites before deploying MedIntel in a production healthcare environment.

Despite these limitations, MedIntel successfully fulfills all objectives defined at the outset of the project. The system provides a functional, tested, and deployable AI-powered health monitoring platform that demonstrates the practical integration of machine learning, large language models, and web technologies in a healthcare context. The project has been successfully tested across all major modules with satisfactory results, and the application is ready for academic submission and can serve as a strong foundation for a production-grade healthcare system with further enhancements.

Chapter 7 Future Scope

While MedIntel provides a comprehensive set of features for academic demonstration, several enhancements can make it production-ready. The current architecture has been deliberately designed with extensibility in mind — the modular Flask blueprint structure, the separation of ML inference from routing logic, and the use of a standard ORM-backed database all provide clean integration points for the features described below.

- **Telemedicine Integration:** Add real-time video consultation between patients and doctors using WebRTC or a third-party API like Twilio. This would eliminate the need for in-person visits for follow-up consultations, making MedIntel a complete end-to-end virtual healthcare platform. The existing appointment management module provides a natural entry point for initiating video calls upon appointment confirmation.
- **Wearable Device Integration:** Connect with smartwatches and fitness bands (e.g., Fitbit, Apple Watch) to auto-fetch real-time vitals like heart rate and SpO2. Automated vital ingestion would replace manual data entry, significantly improving data accuracy and enabling continuous health monitoring rather than point-in-time assessments. The Health Score Engine is already designed to accept structured vital parameters, making wearable integration a relatively straightforward extension.
- **Advanced ML Models**: Replace current models with deep learning approaches (LSTM, Transformer-based) for improved prediction accuracy on larger, certified medical datasets. LSTM-based models would be particularly beneficial for longitudinal health data, enabling the system to detect deteriorating health trends over time rather than evaluating each submission independently. Training on certified datasets such as MIMIC-III would also improve model reliability across diverse patient demographics.
- **RAG-Based Medical Knowledge Base:** Implement a Retrieval-Augmented Generation (RAG) pipeline backed by a medical vector database (PubMed, MedlinePlus) for evidence-based chatbot responses. The current LLM chatbot relies solely on the model's pre-trained knowledge for health guidance. A RAG implementation would ground all responses in peer-reviewed medical literature, significantly improving response accuracy, reliability, and clinical relevance — a critical requirement for any production healthcare application.
- **Mobile Application:** Develop native Android/iOS apps using React Native or Flutter for broader accessibility. A dedicated mobile application would enable push notifications for appointment reminders, health score alerts, and prescription updates, improving patient engagement and medication adherence — key outcomes identified in the literature review.
- **Multilingual Support:** Add support for Hindi and other regional Indian languages to improve accessibility for rural populations. Given that a significant portion of India's population is not proficient in English, multilingual support would dramatically expand

MedIntel's reach and impact, aligning with the system's core goal of democratizing health monitoring.

- **ABDM Integration:** Integrate with India's Ayushman Bharat Digital Mission (ABDM) Health ID system for standardized electronic health records (EHR). ABDM integration would allow MedIntel to participate in India's national digital health ecosystem, enabling seamless health record sharing across hospitals, clinics, and healthcare providers with patient consent.
- **Prescription OCR:** Allow patients to upload paper prescriptions which are digitized using OCR technology. This feature would bridge the gap between traditional paper-based healthcare and MedIntel's digital prescription system, allowing patients who receive handwritten prescriptions from external doctors to maintain a complete digital health record within the platform.
- **Blockchain for Health Records:** Implement blockchain-based immutable health record storage to ensure privacy, security, and patient data ownership. A blockchain-backed storage layer would provide cryptographic guarantees of data integrity and enable patients to selectively share their health records with healthcare providers without surrendering ownership — addressing one of the most significant trust barriers in digital health adoption.
- **Predictive Analytics Dashboard:** Add population-level health analytics for admin to identify trends and at-risk patient groups. Aggregated, anonymized health data from the platform could be used to generate insights such as regional disease prevalence, seasonal health score trends, and high-risk demographic segments — providing actionable intelligence for public health planning and resource allocation.

Collectively, these enhancements would transform MedIntel from an academic demonstration project into a clinically viable, regulatory-compliant, and nationally integrated digital health platform — demonstrating the long-term potential of the architectural and technological choices made during this project.

Bibliography

[1] P. Rajpurkar et al., "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning," arXiv preprint arXiv:1711.05225, 2017.

[2] Z. Obermeyer and E. J. Emanuel, "Predicting the Future — Big Data, Machine Learning, and Clinical Medicine," New England Journal of Medicine, vol. 375, no. 13, pp. 1216–1219, 2016.

[3] A. Esteva et al., "Dermatologist-level classification of skin cancer with deep neural networks," Nature, vol. 542, pp. 115–118, 2017.

[4] L. Laranjo et al., "Conversational agents in healthcare: a systematic review," Journal of the American Medical Informatics Association, vol. 25, no. 9, pp. 1248–1258, 2018.

[5] World Health Organization, "Universal health coverage," WHO Report, 2021. [Online]. Available: https://www.who.int/health-topics/universal-health-coverage. [Accessed: Apr. 2026].

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, 2016.

[8] Flask Documentation, Pallets Projects, 2024. [Online]. Available: https://flask.palletsprojects.com/. [Accessed: Apr. 2026].

[9] Groq API Documentation, Groq Inc., 2024. [Online]. Available: https://console.groq.com/docs. [Accessed: Apr. 2026].

[10] UCI Machine Learning Repository – Heart Disease Dataset. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/heart+disease. [Accessed: Apr. 2026].

[11] IBM Watson Health, IBM Corporation. "Watson for Oncology." IBM Research and Healthcare Division, 2022. [Online]. Available: https://www.ibm.com/watson-health. [Accessed: Apr. 2026].

[12] Buoy Health, Inc. "Buoy Health – AI Symptom Checker and Health Guidance." 2023. [Online]. Available: https://www.buoyhealth.com. [Accessed: Apr. 2026].

[13] HealthTap, Inc. "HealthTap – Virtual Primary Care and AI Health Assistant." 2023. [Online]. Available: https://www.healthtap.com. [Accessed: Apr. 2026].

[14] UCI Machine Learning Repository – Pima Indians Diabetes Dataset. [Online]. Available: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database. [Accessed: Apr. 2026].